CHAPTER 14

# Recursive Definitions

*Notes By Pr. El Hadiq Zouhair*

## 14.1 The recursive idea

A *recursive definition* specifies an object in terms of smaller instances of itself, anchored by explicit base cases. Three kinds of objects are defined this way: *functions* (a base value plus a recurrence), *sets* (base elements plus closure rules), and *structures* (atomic structures plus constructors). Induction (Chapter 13) is the proof principle that matches each kind.

## 14.2 Recursive functions

**Definition 14.1.** A function on $\mathbb{N}$ is *recursively defined* by giving $f(0)$ (and possibly further initial values) and a rule computing $f(n+1)$ from earlier values.

**Example 14.2.** Factorial: $0! = 1$, $(n+1)! = (n+1) \cdot n!$. Powers: $a^0 = 1$, $a^{n+1} = a \cdot a^n$. Fibonacci: $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$. Ackermann's function — recursive in two variables — grows faster than any primitive recursive function.

### 14.2.1 From recurrence to closed form

For *linear homogeneous* recurrences, the characteristic-equation method of Chapter 9 applies: $a_n = c_1 a_{n-1} + c_2 a_{n-2}$ has solution $\alpha r_1^n + \beta r_2^n$ (distinct roots) or $(\alpha + \beta n) r^n$ (double root); Binet's formula $F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}$ is the showcase. Simple non-homogeneous patterns are handled by *unrolling* (iteration) or *telescoping*.

**Example 14.3** (Telescoping)**.** $a_1 = 1$, $a_n = a_{n-1} + (2n-1)$. Unroll: $a_n = 1 + \sum_{k=2}^{n} (2k-1) = \sum_{k=1}^{n} (2k-1) = n^2$ (Chapter 13, Exercise 13.1).

**Example 14.4** (Tower of Hanoi)**.** Moving $n$ discs requires $H_1 = 1$, $H_n = 2H_{n-1} + 1$ moves: move $n-1$ discs aside, the largest disc, then the $n-1$ again. Unrolling, or induction, gives

$H_n = 2^n - 1$. With 64 discs: $2^{64} - 1 \approx 1.8 \times 10^{19}$ moves — the legend's monks will not finish soon.

## 14.3 Recursively defined sets

**Definition 14.5.** A set $S$ is defined recursively by: a *basis* (specified elements of $S$), *recursive rules* (ways to build new elements from old), and the implicit *exclusion* clause: nothing else is in $S$ ($S$ is the *smallest* set satisfying basis and rules).

**Example 14.6** (Even naturals)**.** Basis: $0 \in E$. Rule: $x \in E \Rightarrow x + 2 \in E$. Then $E = \{0, 2, 4, \ldots\}$, the even naturals.

**Example 14.7** (Strings)**.** The set $\Sigma^*$ of strings over alphabet $\Sigma$: basis — the empty string $\lambda \in \Sigma^*$; rule — if $w \in \Sigma^*$ and $a \in \Sigma$ then $wa \in \Sigma^*$. Concatenation, length and reversal are then defined recursively on this structure, e.g. $\text{len}(\lambda) = 0$, $\text{len}(wa) = \text{len}(w) + 1$.

**Example 14.8.** The well-formed formulas of Chapter 2 are a recursively defined set: atoms as basis, connectives as rules — which is what makes structural induction on formulas legitimate.

## 14.4 Recursively defined structures

**Definition 14.9.** *Rooted binary trees*: basis — the empty tree; rule — a root with left and right binary subtrees forms a binary tree. *Full* binary trees: basis — a single vertex; rule — a root joined to two full binary subtrees.

## 14.5 Structural induction

**Theorem 14.10** (Structural induction)**.** To prove that every element of a recursively defined set has property $P$: prove $P$ for each basis element, and prove that each rule preserves $P$ (if the inputs satisfy $P$, so does the output).

**Example 14.11** (Complete binary trees)**.** The *complete* full binary tree of height $n \geq 1$ has $2^n - 1$ vertices, of which $2^{n-1}$ are leaves. Induction on $n$: height 1 — one vertex, one leaf: $2^1 - 1 = 1$, $2^0 = 1$. Step: the tree of height $n + 1$ is a root over two trees of height $n$: $2(2^n - 1) + 1 = 2^{n+1} - 1$ vertices and $2 \cdot 2^{n-1} = 2^n$ leaves.

**Example 14.12** (Strings)**.** $\text{len}(uv) = \text{len}(u) + \text{len}(v)$: structural induction on $v$. Base $v = \lambda$: $\text{len}(u\lambda) = \text{len}(u)$. Rule case $v = wa$: $\text{len}(u(wa)) = \text{len}((uw)a) = \text{len}(uw) + 1 = \text{len}(u) + \text{len}(w) + 1 = \text{len}(u) + \text{len}(wa)$.

## 14.6 Recursion in computation

### 14.6.1 Recursive algorithms and correctness

A recursive algorithm mirrors a recursive definition; induction proves it correct. For the recursive factorial: base — the program returns $1 = 0!$ on input 0; step — if it returns $n!$ on $n$, then on $n+1$ it returns $(n+1) \cdot n! = (n+1)!$. Mergesort splits a list in half, sorts each recursively and merges; strong induction on length proves correctness, and its running time obeys $T(n) = 2T(n/2)+O(n)$, solved by the *master theorem* as $T(n) = O(n \log n)$.

### 14.6.2 Memoization and dynamic programming

Naive recursive Fibonacci recomputes subproblems exponentially ($T(n) = T(n-1) + T(n-2) + O(1)$ is itself Fibonacci-sized). Caching each computed value — *memoization* — or filling a table bottom-up — *dynamic programming* — reduces the cost to linear. The lesson: a recursive *definition* need not dictate a naive recursive *computation*.

### 14.6.3 Tail recursion, mutual recursion, fixed points

A *tail-recursive* function performs its recursive call last, so it can be transformed mechanically into a loop (accumulator pattern): $\text{fact}(n, acc) = \text{fact}(n-1, n \cdot acc)$. *Mutual* recursion defines functions in terms of each other, e.g. $\text{even}(0) = \texttt{T}$, $\text{even}(n) = \text{odd}(n-1)$; $\text{odd}(0) = \texttt{F}$, $\text{odd}(n) = \text{even}(n-1)$ — proved correct by simultaneous induction. More abstractly, a recursive definition presents its object as a *fixed point* of an operator ($S = F(S)$); self-similar fractals such as the Koch curve and Sierpiński triangle, and the generations of a cellular automaton (e.g. rule 90, which paints Pascal's triangle mod 2), are recursive definitions made visible.

> *Common recursion pitfalls: a missing or unreachable base case (infinite regress); recursive calls that do not strictly decrease toward the base; overlapping subproblems solved exponentially often when a cache would do; and confusing the definition of a set with membership testing — the exclusion clause matters.*

## 14.7 Summary

Recursive definitions build functions (base values + recurrence), sets (basis + closure rules + exclusion) and structures (atoms + constructors). Closed forms come from characteristic equations, unrolling or telescoping; structural induction is the universal proof method, with weak/strong induction as its special case on $\mathbb{N}$. Computationally, recursion pairs with induction for correctness, with memoization for efficiency, and with iteration via tail calls.

## Exercises

**Exercise 14.1** EASY

Give recursive definitions of: (a) $f(n) = 5n$; (b) $g(n) = 3^n$; (c) the sequence $4, 9, 14, 19, \ldots$

**Exercise 14.2** EASY

Compute $a_4$ where $a_0 = 3$ and $a_n = 2a_{n-1} - 1$, then conjecture and verify a closed form.

### Exercise 14.3 EASY

Define recursively the set $S = \{3, 6, 9, 12, \ldots\}$ of positive multiples of 3.

### Exercise 14.4 MEDIUM

Solve by unrolling: $a_1 = 2$, $a_n = a_{n-1} + 2^n$, and verify by induction.

### Exercise 14.5 MEDIUM

Prove by induction that the Tower of Hanoi recurrence $H_1 = 1$, $H_n = 2H_{n-1} + 1$ has solution $H_n = 2^n - 1$.

### Exercise 14.6 MEDIUM

Define the reversal of a string by $\lambda^R = \lambda$ and $(wa)^R = a\,w^R$. Prove by structural induction that $(uv)^R = v^R u^R$.

### Exercise 14.7 MEDIUM

Let $S$ be defined by: $5 \in S$; if $x, y \in S$ then $x + y \in S$. Determine $S$ exactly and prove your answer by structural induction (one inclusion) and ordinary argument (the other).

### Exercise 14.8 MEDIUM

A *full* binary tree is a single vertex, or a root with two full binary subtrees. Prove by structural induction that the number of leaves of a full binary tree exceeds the number of internal vertices by exactly one.

### Exercise 14.9 HARD

Define $T(n) = T(\lfloor n/2 \rfloor) + 1$, $T(1) = 1$. Show $T(n) = \lfloor \log_2 n \rfloor + 1$ by strong induction.

### Exercise 14.10 HARD

Naive recursive Fibonacci makes $C_n$ function calls, where $C_0 = C_1 = 1$ and $C_n = C_{n-1} + C_{n-2} + 1$. Prove $C_n = 2F_{n+1} - 1$, confirming exponential blow-up.

### Exercise 14.11 HARD

The Koch curve is defined recursively: $K_0$ is a segment of length 1; $K_{n+1}$ replaces each segment of $K_n$ by four segments each one third as long. Find the length $L_n$ of $K_n$, prove it by induction, and conclude about $\lim L_n$.

## Solutions to the Exercises

**Solution 14.1.** (a) $f(0) = 0$, $f(n+1) = f(n) + 5$. (b) $g(0) = 1$, $g(n+1) = 3\,g(n)$. (c) $a_0 = 4$, $a_{n+1} = a_n + 5$. ■

**Solution 14.2.** $a_1 = 5$, $a_2 = 9$, $a_3 = 17$, $a_4 = 33$. Conjecture $a_n = 2^{n+1} + 1$: base $a_0 = 3$; step $2(2^{n+1} + 1) - 1 = 2^{n+2} + 1$. ✓ ■

**Solution 14.3.** Basis: $3 \in S$. Rule: $x \in S \Rightarrow x + 3 \in S$. (Exclusion clause: nothing else.) An easy induction shows $S = \{3k \mid k \geq 1\}$. ■

**Solution 14.4.** $a_n = 2 + \sum_{k=2}^{n} 2^k = 2 + (2^{n+1} - 4) = 2^{n+1} - 2$. Induction: base $a_1 = 2 = 2^2 - 2$; step $a_{n+1} = (2^{n+1} - 2) + 2^{n+1} = 2^{n+2} - 2$. ■

**Solution 14.5.** Base: $H_1 = 1 = 2^1 - 1$. Step: $H_{n+1} = 2H_n + 1 = 2(2^n - 1) + 1 = 2^{n+1} - 1$. ■

**Solution 14.6.** Structural induction on $v$. Base $v = \lambda$: $(u\lambda)^R = u^R = \lambda^R u^R$. Step $v = wa$: $(u(wa))^R = ((uw)a)^R = a\,(uw)^R = a\,(w^R u^R) = (a\,w^R)\,u^R = (wa)^R u^R$. ■

**Solution 14.7.** $S = \{5k \mid k \geq 1\}$, the positive multiples of 5. ($\subseteq$) Structural induction: the basis element 5 is a multiple of 5, and the sum of two multiples of 5 is one. ($\supseteq$) Induction on $k$: $5 \in S$; if $5k \in S$ then $5(k+1) = 5k + 5 \in S$ by the rule. ■

**Solution 14.8.** $P(T)$: leaves$(T)$ = internal$(T)+1$. Base: a single vertex has 1 leaf, 0 internal: ✓. Step: $T$ = root over $T_1, T_2$ satisfying $P$. Then leaves$(T)$ = leaves$(T_1)$ + leaves$(T_2)$ = internal$(T_1)$ + internal$(T_2)$ + 2 = internal$(T)$ + 1, since internal$(T)$ = internal$(T_1)$ + internal$(T_2)$ + 1 (the new root). ■

**Solution 14.9.** $P(n)$: $T(n) = \lfloor \log_2 n \rfloor + 1$. Base: $T(1) = 1 = 0 + 1$. Step: let $n \geq 2$, so $1 \leq \lfloor n/2 \rfloor < n$; by strong hypothesis $T(n) = \lfloor \log_2 \lfloor n/2 \rfloor \rfloor + 2$. Writing $2^k \leq n < 2^{k+1}$ (so $\lfloor \log_2 n \rfloor = k$), we get $2^{k-1} \leq \lfloor n/2 \rfloor < 2^k$, whence $\lfloor \log_2 \lfloor n/2 \rfloor \rfloor = k - 1$ and $T(n) = (k-1) + 2 = k + 1$. ✓ ■

**Solution 14.10.** Induction with two bases: $C_0 = 1 = 2F_1 - 1$, $C_1 = 1 = 2F_2 - 1$. Step: $C_n = C_{n-1} + C_{n-2} + 1 = (2F_n - 1) + (2F_{n-1} - 1) + 1 = 2F_{n+1} - 1$. Since $F_{n+1} \sim \varphi^{n+1}/\sqrt{5}$, the call count grows exponentially in $n$. ■

**Solution 14.11.** Each step multiplies the number of segments by 4 and divides their length by 3, so $L_n = (4/3)^n$. Induction: $L_0 = 1$; step $L_{n+1} = 4 \cdot \frac{L_n}{3} \cdot \frac{\text{(per segment)}}{\text{(same)}} = \frac{4}{3} L_n = (4/3)^{n+1}$. As $4/3 > 1$, $L_n \to \infty$: the limiting curve has infinite length (while spanning a bounded region — the hallmark of a fractal). ■